

DBC 解析模块接口使用手册

CAN 接口卡系列产品

TN01010101

使用手册

类别	内容
关键词	CAN、DBC 解析与收发、多帧传输
摘要	本文档用于说明 DBC 解析库的接口、使用方法以及注意 点。

修订历史

版本	日期	原因
V1.00	2017/04/01	创建文档
V1.01	2018/07/31	添加信号的值与含义对解析
V1.02	2018/11/12	更新信号结构
V1.03	2020/04/21	去除仿真部分
V1.04	2021/03/15	支持 CANFD，去掉信号结构体中的实际值

目录

1. 功能简介.....	1
2. 接口说明及其使用.....	2
2.1 结构体说明.....	2
2.1.1 发送错误码.....	2
2.1.2 字符串长度.....	2
2.1.3 复用器类型.....	2
2.1.4 DBCSignal	2
2.1.5 DBCMessage.....	3
2.1.6 FileInfo	3
2.1.7 ValDescPair.....	3
2.1.8 数据帧结构类型.....	3
2.2 回调函数.....	4
2.2.1 OnSend.....	4
2.2.2 OnMultiTransDone	4
2.2.3 OnAsyncAnalyseDone	4
2.3 接口库函数说明.....	4
2.3.1 ZDBC_SetLogFile	4
2.3.2 ZDBC_SetFrameType	5
2.3.3 ZDBC_Init.....	5
2.3.4 ZDBC_Release.....	5
2.3.5 ZDBC_LoadFile	5
2.3.6 ZDBC_SetProtocol.....	6
2.3.7 ZDBC_GetFirstMessage	6
2.3.8 ZDBC_GetNextMessage.....	6
2.3.9 ZDBC_GetMessageById	6
2.3.10 ZDBC_GetMessageCount.....	7
2.3.11 ZDBC_Analyse	7
2.3.12 ZDBC_AsyncAnalyse	7
2.3.13 ZDBC_OnReceive	8
2.3.14 ZDBC_SetSender	8
2.3.15 ZDBC_SetCANFDSEnder	8
2.3.16 ZDBC_SetOnMultiTransDoneFunc.....	9
2.3.17 ZDBC_Send	9
2.3.18 ZDBC_GetValDescPairCount.....	9
2.3.19 ZDBC_GetValDescPair.....	10
2.3.20 ZDBC_Decode	10
2.3.21 ZDBC_Encode	10
2.3.22 ZDBC_CalcActualValue	11
2.3.23 ZDBC_CalcRawValue	11
2.4 接口库函数使用流程.....	12
3. 例程使用.....	13

3.1	程序主界面.....	13
4.	免责声明.....	14

1. 功能简介

本文档用于介绍 DBC 解析库 ZDBC，先介绍模块中各接口的含义和具体的使用方式、再讲解 demo 使用方法。

ZDBC 库包含三大功能：

1. 解析 DBC 文件：解析并提供接口获取文件中的所有消息和信号信息；
2. 解析帧数据：根据帧数据和消息定义，返回消息和信号信息；
3. 数据收发：用户只需要填充消息结构以及信号值，并调用模块接口即可实现消息的发送，不必关心如何根据 DBC 定义去计算如何填充数据；除了单帧数据发送，还实现了多帧收发处理，库内部已经实现了多帧发送机制，用户像使用单帧一样即可实现多帧发送，对于多帧接收，库内部已经实现了应答机制，用户不必做任何操作，收发的多帧数据都会通过回调接口返回给用户。

2. 接口说明及其使用

2.1 结构体说明

2.1.1 发送错误码

以下宏定义了发送函数 ZDBC_Send 的返回值。

```
#define ERR_SUCCESS          0
#define ERR_FAILED          1
#define ERR_MULTI_TRANSMITTING 2 //多帧传输发送中
#define ERR_NOT_SUPPORT     3 //功能不支持,或未开启
```

2.1.2 字符串长度

以下宏定义了各类字符串的最大长度,如果实际长度超过了最大限制则被截断,不过一般情况下已经足够使用。

```
#define _MAX_FILE_PATH_      260 //最长文件路径
#define _DBC_NAME_LENGTH_   127 //名称最长长度
#define _DBC_COMMENT_MAX_LENGTH_ 200 //注释最长长度
#define _DBC_UNIT_MAX_LENGTH_ 10 //单位最长长度
#define _DBC_SIGNAL_MAX_COUNT_ 128 //一个消息含有的信号的最大数目
```

2.1.3 复用器类型

```
#define MUTIPLEXER_NONE 0 //不使用复用器
#define MUTIPLEXER_M_VALUE 1 //复用信号,当复用器开关的值为 multiplexer_value 时,该信号有效
#define MUTIPLEXER_M_SWITCH 2 //复用器开关,一个 DBC 消息只能有一个信号为开关
```

2.1.4 DBCSignal

以下结构 DBCSignal 定义了一个信号,用户着重关注 nValue 项,这个是信号值,作为解析结果返回,也可以修改该值发送出去,一个消息 DBCMessage 可能会包含多个信号。

```
struct DBCSignal
{
    uint32 nStartBit; // 起始位
    uint32 nLen; // 位长度
    double nFactor; // 转换因子
    double nOffset; // 转换偏移实际值=原始值*nFactor+nOffset
    double nMin; // 最小值
    double nMax; // 最大值
    uint64 nRawValue; // 原始值
    bool is_signed; // 1:有符号数据, 0:无符号
    bool is_motorola; // 是否摩托罗拉格式
    uint8 multiplexer_type; // 见 'multiplexer type' above
    uint8 multiplexer_value;
    uint8 val_type; // 0:integer, 1:float, 2:double
    char unit[_DBC_UNIT_MAX_LENGTH_+1]; // 单位
```

```
char strName[_DBC_NAME_LENGTH+1]; //名称
char strComment[_DBC_COMMENT_MAX_LENGTH+1]; //注释
char strValDesc[_DBC_NAME_LENGTH+1]; //值描述;
}
```

2.1.5 DBCMessage

以下结构 DBCMessage 定义了一个消息，消息跟帧 ID 对应，一个 DBC 文件当中含有多个消息。

```
struct DBCMessage
{
    uint32 nSignalCount; //信号数量
    uint32 nID;
    uint32 nSize; //消息占的字节数目
    double nCycleTime; //发送周期
    uint8 nExtend; //1:扩展帧, 0:标准帧
    DBCSignal vSignals[_DBC_SIGNAL_MAX_COUNT_]; //信号集合
    char strName[_DBC_NAME_LENGTH+1]; //名称
    char strComment[_DBC_COMMENT_MAX_LENGTH+1]; //注释
};
```

2.1.6 FileInfo

结构 FileInfo 作为初始化参数，strFilePath 指明 DBC 文件的绝对路径。

```
#define PROTOCOL_J1939 0
#define PROTOCOL_OTHER 1
struct FileInfo
{
    char strFilePath[_MAX_FILE_PATH+1]; //dbc 文件路径
    uint8 merge; //1:不清除现有的数据，即支持加载多个文件；0：清除原来的数据
};
```

2.1.7 ValDescPair

值与含义对，例如"Not Supported" 2 "Error" 1 "Not Defined"。

```
struct ValDescPair
{
    double value;
    char desc[_DBC_NAME_LENGTH+1];
};
```

2.1.8 数据帧结构类型

如果 CAN 通讯库是使用 zlgcan.dll 库，使用 ZDBC_SetFrameType 设置数据类型为 ZDBC_FRAME_TYPE_ZLGCAN，后续接口中的 pObj 根据是否使用 CANFD 传入 can_frame 或 canfd_frame 帧数据结构；

如果 CAN 通讯库是使用 ControlCAN.dll 库，使用 ZDBC_SetFrameType 设置数据类型为 ZDBC_FRAME_TYPE_CONTROLCAN，后续接口中的 pObj 传入 VCI_CAN_OBJ 帧数据结构。

```
#define ZDBC_FRAME_TYPE_ZLGCAN 0 // 使用 zlgcan 库的 can_frame 或者
```

```
canfd_frame
#define ZDBC_FRAME_TYPE_CONTROLCAN 1 // 使用 ControlCAN 的 VCI_CAN_OBJ
```

2.2 回调函数

解析库提供了回调函数对数据做进一步处理或返回处理结果。

2.2.1 OnSend

解析库不做实际的帧数据发送, 把消息打包成帧数据后会调用该回调函数让用户去做实际的发送, 用户只需要在回调函数内部简单的调用通讯库发送接口即可实现发送。回调函数定义如下:

```
typedef void (FUNCCALL *OnSend)(void* ctx, void* pObj);
typedef bool (FUNCCALL *OnSendCANFD)(void* ctx, void* pObj);
```

- **ctx** 是上下文信息, 实际上是通过 **DBC_SetSender** 设置进来的, 就是可以通过 **DBC_SetSender** 设置一些额外的信息, 然后回调函数会回传给用户, 方便做一些其他处理;
- **pObj** 实际上是一个数据帧结构, 参考 2.1.8, 转化为该类型即可。

2.2.2 OnMultiTransDone

由于 CAN 帧数据最大是 8 字节, 超过 8 字节的帧数据需要分包多帧发送, 解析库内部已经实现了多帧收发机制, 用户不用关心实际的处理, 处理结果会通过该回调返回, 如果不关心实际的处理结果也可以不实现该回调。回调函数定义如下:

```
typedef void (*OnMultiTransDone)(void* ctx, DBCMessage* pMsg, uint8* data, uint16 nLen, uint8 nDirection);
```

- **ctx** 是上下文信息, 实际上是通过 **DBC_SetOnMultiTransDoneFunc** 设置进来的, 就是可以通过 **DBC_SetOnMultiTransDoneFunc** 设置一些额外的信息, 然后回调函数会回传给用户, 方便做一些其他处理。
- **pMsg** 是多帧数据对应的消息, 消息包含了所有的信号值;
- **data** 是多帧的实际数据;
- **nLen** 是多帧的数据长度;
- **nDirection** 是传输方向, 1: 接收, 0: 发送

2.2.3 OnAsyncAnalyseDone

采用独立线程异步解析数据帧, 将解析出的 **DBCMessage** 通过此回调函数返回。回调函数定义如下:

```
typedef void (FUNCCALL *OnAsyncAnalyseDone)(void* ctx, DBCMessage* pMsg, uint64 extraData);
```

- **ctx** 是上下文信息, 实际上是通过 **DBC_SetSender** 设置进来的, 就是可以通过 **DBC_SetSender** 设置一些额外的信息, 然后回调函数会回传给用户, 方便做一些其他处理;
- **pMsg** 返回的解析结果;
- **extraData** 请求解析时传入的额外标识信息。

2.3 接口库函数说明

2.3.1 ZDBC_SetLogFile

描述

此函数用于设置日志文件的输出，默认无日志输出。

```
EXTERN_C void FUNCCALL ZDBC_SetLogFile( uint8 enable, const char* filepath );
```

2.3.2 ZDBC_SetFrameType

描述

设置数据帧结构体类型，默认为 ZDBC_FRAME_TYPE_ZLGCAN。

```
EXTERN_C void FUNCCALL ZDBC_SetFrameType( uint32 type );
```

```
EXTERN_C uint32 FUNCCALL ZDBC_GetFrameType();
```

参数

type

数据帧结构类型，根据使用的通讯库选择，可以为 ZDBC_FRAME_TYPE_ZLGCAN 或 ZDBC_FRAME_TYPE_CONTROLCAN。

2.3.3 ZDBC_Init

描述

此函数用于初始化解析模块，只需要初始化一次。

```
EXTERN_C DBCHandle FUNCCALL ZDBC_Init(uint8 disableMultiSend = 0, uint8 enableAsyncAnalyse = 0)
```

参数

disableMultiSend

是否关闭多帧发送，为 1 时不支持多帧的消息发送。

enableAsyncAnalyse

是否开启异步解析；0-不启动，ZDBC_AsyncAnalyse 接口无效；1-启动，独立线程解析出消息。

返回值

为 INVALID_DBC_HANDLE 表示初始化失败，其他表示初始化成功，保存该返回值，之后的函数调用都要用到该句柄。

2.3.4 ZDBC_Release

描述

释放资源，与 ZDBC_Init 配对使用。

```
EXTERN_C void FUNCCALL ZDBC_Release( DBCHandle hDBC );
```

参数

hDBC

句柄，DBC_Init()的返回值，之后提到的句柄都为该值。

2.3.5 ZDBC_LoadFile

描述

此函数用以加载 DBC 格式文件。

```
EXTERN_C bool FUNCCALL ZDBC_LoadFile( DBCHandle hDBC, const FileInfo* pFileInfo );
```

参数

hDBC

句柄；

pFileInfo

文件信息，参考 2.1.6。

返回值

为 true 表示加载成功，false 表示失败。

2.3.6 ZDBC_SetProtocol

描述

设置 DBC 文件协议类型，有些 DBC 文件中指定了协议类型，ZDBC_LoadFile 时已经获取协议类型，通过 ZDBC_GetProtocol 可以获取。如果需要修改，则调用 ZDBC_SetProtocol 进行设置。

```
EXTERN_C void FUNCCALL ZDBC_SetProtocol( DBCHandle hDBC, uint8 type);
EXTERN_C uint8 FUNCCALL ZDBC_GetProtocol( DBCHandle hDBC );
```

参数

hDBC

句柄；

pFileInfo

文件信息，参考 2.1.6。

返回值

为 true 表示加载成功，false 表示失败。

2.3.7 ZDBC_GetFirstMessage

描述

此函数用于获取第一条消息数据。

```
EXTERN_C bool FUNCCALL ZDBC_GetFirstMessage( DBCHandle hDBC, DBCMessage* pMsg );
```

参数

hDBC

句柄；

pMsg

消息信息，结构参考 2.1.5。

返回值

为 true 表示获取成功，false 表示失败。

2.3.8 ZDBC_GetNextMessage

描述

此函数用以获取下一条消息数据，结合 ZDBC_GetFirstMessage 使用。

```
EXTERN_C bool FUNCCALL ZDBC_GetNextMessage( DBCHandle hDBC, DBCMessage* pMsg );
```

参数

hDBC

句柄；

pMsg

消息信息，结构参考 2.1.5。

返回值

为 true 表示获取成功，false 表示失败。

2.3.9 ZDBC_GetMessageById

描述

此函数用以根据 ID 获取消息数据。

```
EXTERN_C bool FUNCCALL ZDBC_GetMessageById( DBCHandle hDBC, uint nID, DBCMessage* pMsg );
```

参数

hDBC

句柄;

nID

帧 ID;

pMsg

消息信息, 结构参考 2.1.5。

返回值

为 true 表示获取成功, false 表示失败。

2.3.10 ZDBC_GetMessageCount

描述

此函数用以获取 DBC 文件中含有的消息数目。

```
EXTERN_C uint __stdcall ZDBC_GetMessageCount( DBCHandle hDBC );
```

参数

hDBC

句柄。

返回值

DBC 文件中含有的消息数目

2.3.11 ZDBC_Analyse

描述

此函数用以解析帧数据, 返回解析结果。

```
EXTERN_C bool ZDBC_Analyse( DBCHandle hDBC, const void* pObj, DBCMessage* pMsg );
```

参数

hDBC

句柄;

pObj

VCI_CAN_OBJ 指针, 参考[错误!未找到引用源。](#)。

pMsg

消息信息, 结构参考 2.1.5。

返回值

为 true 表示解析成功, false 表示失败。

2.3.12 ZDBC_AsyncAnalyse

描述

此函数用以独立线程异步解析帧数据, 返回解析结果。使用此功能, 必须在 ZDBC_Init 时 enableAsyncAnalyse 参数传入 1 来启动该功能。

```
EXTERN_C bool FUNCCALL ZDBC_AsyncAnalyse(DBCHandle hDBC, const void* pObj, uint8 frame_type, uint64 extraData);
```

```
EXTERN_C void FUNCCALL ZDBC_SetAsyncAnalyseCallback( DBCHandle hDBC, OnAsyncAnalyseDone
```

```
cb, void* ctx);
```

参数

hDBC

句柄;

pObj

数据结构体指针, 参考 2.1.8。

frame_type

帧类型, 参考 FT_CAN、FT_CANFD, ControlCAN 库不支持 CANFD。

extraData

此帧数据的附带数据, 解析完成后会回调传回此参数; 例如帧时间戳。

返回值

为 true 表示解析成功, false 表示失败。

2.3.13 ZDBC_OnReceive

描述

用户需要调用该函数把接收到的帧数据传进来, 涉及多帧传输必须要调用, 否则无法实现报文交互, 可以实现为接收到每一个帧都调用该函数一次。

```
EXTERN_C void FUNCCALL ZDBC_OnReceive( DBCHandle hDBC, const void* pObj );
```

参数

hDBC

句柄;

pObj

数据结构体指针, 参考 2.1.8。

2.3.14 ZDBC_SetSender

描述

此函数用以设置实际发送数据的回调函数, 涉及数据发送时必须设置, 只需要设置一次。

```
EXTERN_C void FUNCCALL ZDBC_SetSender( DBCHandle hDBC, OnSend sender, void* ctx );
```

参数

hDBC

句柄;

sender

回调函数, 参考 2.2.1。

ctx

上下文, 可以是任意指针, 作为回调函数的回传参数。

2.3.15 ZDBC_SetCANFDSEnder

描述

此函数用以设置实际发送 CANFD 数据的回调函数, 涉及数据发送时必须设置, 只需要设置一次。

```
EXTERN_C void FUNCCALL ZDBC_SetCANFDSEnder( DBCHandle hDBC, OnSendCANFD sender, void* ctx );
```

参数

hDBC

句柄;

sender

回调函数，参考 2.2.1。

ctx

上下文，可以是任意指针，作为回调函数的回传参数。

2.3.16 ZDBC_SetOnMultiTransDoneFunc

描述

设置处理多帧传输数据的回调函数，只需要设置一次，多帧传输结束后会调用该回调函数返回解析结果，如果不想处理多帧结果，可以不设置该回调函数。

```
EXTERN_C void ZDBC_SetOnMultiTransDoneFunc( DBCHandle hDBC, OnMultiTransDone func, void* ctx);
```

参数

hDBC

句柄；

func

回调函数，参考 2.2.2。

ctx

上下文，可以是任意指针，作为回调函数的回传参数。

2.3.17 ZDBC_Send

描述

发送 DBC 消息。

```
EXTERN_C ERR_CODE FUNCCALL ZDBC_Send( DBCHandle hDBC, const DBCMessage* pMsg, uint8 frame_type );
```

参数

hDBC

句柄；

pMsg

消息信息，结构参考 2.1.5。

frame_type

使用 CAN 或者 CANFD 帧发送，参考 FT_CAN、FT_CANFD。

返回值

参考发送错误码。

2.3.18 ZDBC_GetValDescPairCount

描述

获取具体信号的值与含义对个数，值与含义例如"Not Supported" 2 "Error" 1 "Not Defined"。

```
EXTERN_C uint32 __stdcall ZDBC_GetValDescPairCount( DBCHandle hDBC, uint32 msg_id, char* signal_name);
```

参数

hDBC

句柄；

msg_id

消息 ID。

signal_name

信号名

返回值

值与含义对个数。

2.3.19 ZDBC_GetValDescPair

描述

获取具体信号的值与含义对，请先使用 ZDBC_GetValDescPairCount 获取个数。

```
EXTERN_C void __stdcall ZDBC_GetValDescPair( DBCHandle hDBC, uint32 msg_id, char* signal_name, ValDescPair* pair);
```

参数

hDBC

句柄；

msg_id

消息 ID。

signal_name

信号名

pair

存放返回的值与含义对的缓冲区，大小由用户构造，强烈建议设置为 ZDBC_GetValDescPairCount 返回值大小。

2.3.20 ZDBC_Decode

描述

根据原始数据解码为 DBCMessage。

```
EXTERN_C bool FUNCCALL ZDBC_Decode( DBCHandle hDBC, DBCMessage* pMsg, const void* pObj, uint32 nCount );
```

参数

hDBC

句柄；

pMsg

输出参数，解析结果。

pObj

帧数据数组, ControlCAN 传入 VCI_CAN_OBJ, zlgcan 传入 can_frame。

nCount

原始帧数据个数，即数组大小。

返回值

是否成功。

2.3.21 ZDBC_Encode

描述

根据 DBCMessage 编码为原始数据。

```
EXTERN_C bool FUNCCALL ZDBC_Encode( DBCHandle hDBC, void* pObj, uint32* nCount, const DBCMessage* pMsg );
```

参数

hDBC

句柄;

pObj

编码的原始数据缓冲区数组, ControlCAN 传入 VCI_CAN_OBJ, zlgcan 传入 can_frame。

nCount

输出参数, *pObj* 缓冲区大小, 返回时为实际原始数据个数。

pMsg

输入参数, DBC 消息。

返回值

是否成功。

2.3.22 ZDBC_CalcActualValue

描述

信号原始值转换为实际值。

```
EXTERN_C double FUNCCALL ZDBC_CalcActualValue(const DBCSignal* sgl, uint64 *rawVal);
```

参数

sgl

DBC 信号;

rawVal [in,out]

原始值, 如果该值超出信号长度可表示范围, 会被截断。

返回值

实际值。

2.3.23 ZDBC_CalcRawValue

描述

信号实际值转换为原始值。

```
EXTERN_C uint64 FUNCCALL ZDBC_CalcRawValue(const DBCSignal* sgl, double *actualVal);
```

参数

sgl

DBC 信号;

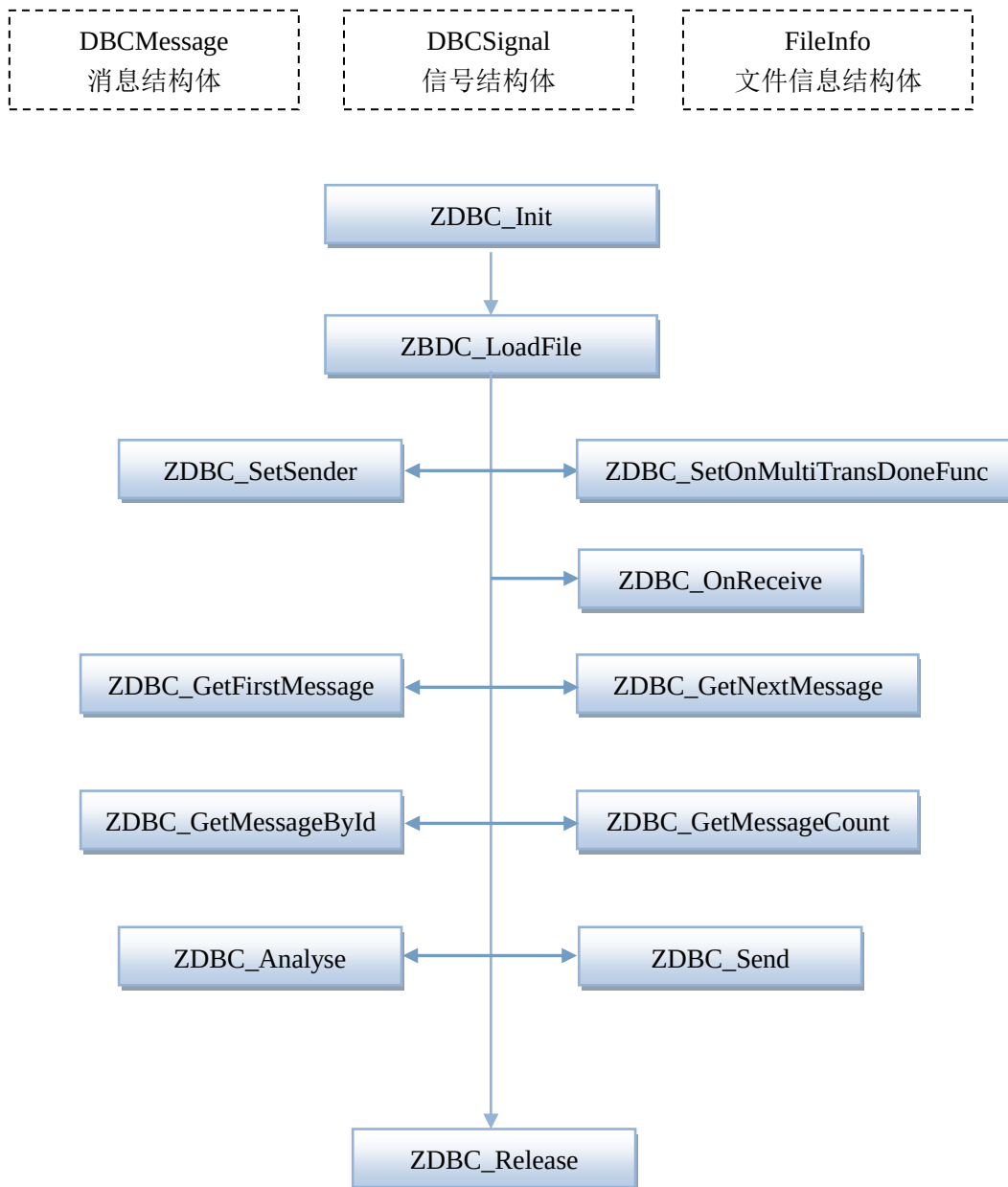
actualVal [in,out]

实际值, 超出可表示范围时会被修正。

返回值

原始值。

2.4 接口库函数使用流程



3. 例程使用

为了更加清晰明了的展示库函数接口的使用，提供了一个基于 Visual Studio 2008 的 VC++例程，随库文件一起发布。

3.1 程序主界面

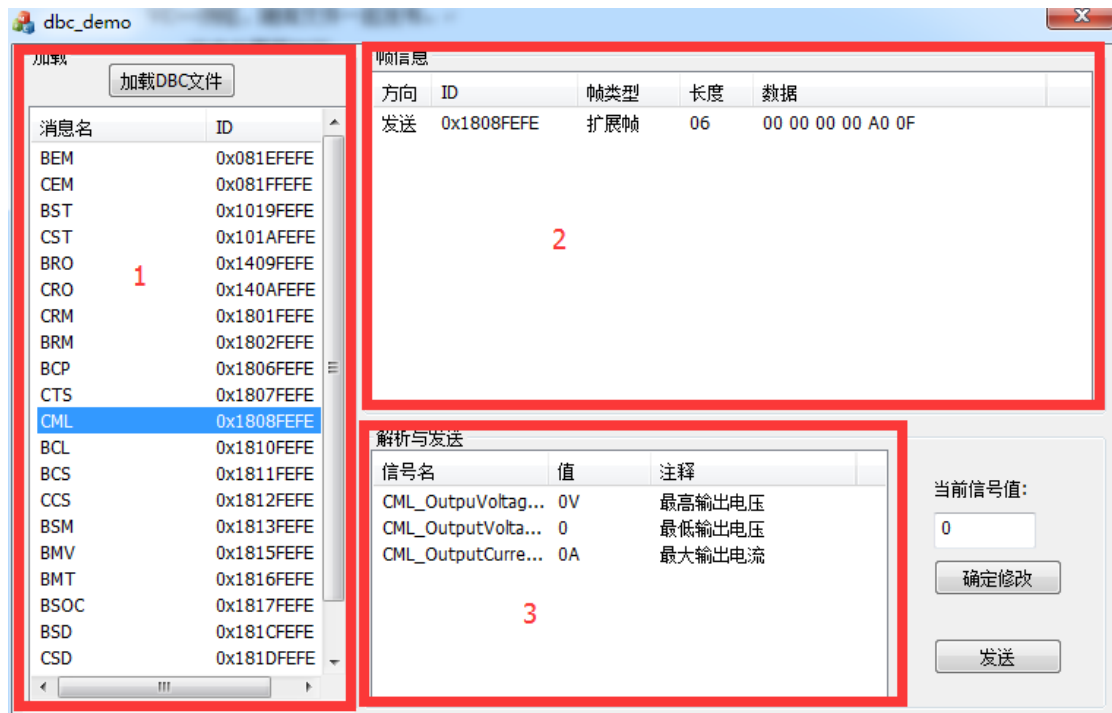


图 1 demo 主界面

1. 区域 1 显示了 DBC 文件当中所有的消息，点击消息可以在区域 3 中看到该消息的所有信号；
2. 区域 2 显示收发的帧数据，点击帧数据可以在区域 3 看到解析结果；
3. 区域 3 显示当前消息的所有信号以及信号值，选中某一信号，可以在右侧对信号值进行修改，点击“确认修改”修改生效；

4. 免责声明

广州致远电子有限公司隶属于广州立功科技股份有限公司。本着为用户提供更好服务的原则，广州致远电子有限公司（下称“致远电子”）在本手册中将尽可能地向用户呈现详实、准确的产品信息。但鉴于本手册的内容具有一定的时效性，致远电子不能完全保证该文档在任何时段的时效性与适用性。致远电子有权在没有通知的情况下对本手册上的内容进行更新，恕不另行通知。为了得到最新版本的信息，请尊敬的用户定时访问致远电子官方网站或者与致远电子工作人员联系。感谢您的包容与支持！

销售与服务网络

广州致远电子有限公司

地址：广州市天河区车陂路黄洲工业区 7 栋 2 楼
 邮编：510660
 网址：www.zlg.cn



全国服务热线电话：400-888-4005

广州总公司

广州市天河区车陂路黄洲工业区 7 栋 2 楼

上海分公司

上海市北京东路 668 号科技京城东楼 12E 室

北京分公司

北京市丰台区马家堡路 180 号 蓝光云鼎 208 室

深圳分公司

深圳市宝安区新安街道海秀路 21 号龙光世纪大厦 A 座 1205

武汉分公司

武汉市洪山区民族大道江南家园 1 栋 3 单元 602 室

南京分公司

南京市秦淮区汉中路 27 号友谊广场 17 层 F、G 区

杭州分公司

杭州市西湖区紫荆花路 2 号杭州联合大厦 A 座 4 单元 508 室

成都分公司

四川省成都市高新技术开发区天府大道中段 500 号东方希望天祥广场 1 栋 C 座 3521 室（地铁世纪城站 B 出口）

郑州分公司

河南省郑州市中原区建设西路 118 号 1 号楼 3 单元 13 层 1302 室（华亚广场）

重庆分公司

重庆市渝北区龙溪街道新溉大道 18 号山顶国宾城 11 幢 4-14

西安办事处

西安市长安北路 54 号太平洋大厦 1201 室

天津办事处

天津市河东区津塘路与十一经路交口鼎泰大厦 1004

青岛办事处

山东省青岛市李沧区枣园路 11 号银座华府 1 号楼 2 单元 1901 室